

Quantization, Mixed Precision, and LLM System Costs

Lecturer: Hao Zhang

*Scribe: Varad Kottawar, Aniruddha Fale, Jonas Tsai,
Vincent Chen, ChengYang Wu, Meng-Chi Tsai, Raghur-
rinivasan Venkatesan, Manasvin Surya B J, Jason Wang*

1 Overview

This lecture connects three system-level themes in modern machine learning systems: quantization, mixed-precision training, and the computational structure of large language models (LLMs). Quantization reduces storage and memory bandwidth by representing weights or activations with fewer bits. Mixed precision chooses different numerical formats for different operations, keeping precision-sensitive operations stable while accelerating the heavy matrix multiplications. Finally, Transformer-based LLMs expose several recurring bottlenecks: compute, memory, and communication.

2 Recap: K-Means Quantization

K-means quantization compresses a weight tensor by clustering its values and representing each value using the index of its nearest centroid. The procedure is:

1. Run k-means clustering on the floating-point weight values to obtain a small set of centroids.
2. Replace each floating-point weight with the integer index of its nearest centroid.
3. Store both the integer index tensor and the floating-point codebook of centroids.

This method reduces storage because the tensor stores low-bit indices instead of full-precision values. However, it does not directly reduce computation. During inference, arithmetic is still performed on dequantized floating-point values, so the matrix multiplication itself does not receive an integer-arithmetic speedup.

3 Linear Quantization

3.1 Geometric Interpretation

Linear quantization maps a continuous floating-point interval to a discrete integer interval. If the floating-point range is

$$[r_{\min}, r_{\max}],$$

and the target quantized range is

$$[q_{\min}, q_{\max}],$$

then quantization linearly maps the first interval onto the second. For example, 2-bit signed quantization has four representable integer values and can use the range

$$q_{\min} = -2, \quad q_{\max} = 1.$$

3.2 Scale and Zero Point

Two parameters define the mapping:

- **Scale** S : a floating-point step size for the quantization grid.
- **Zero point** Z : an integer offset that aligns real zero with a representable quantized value.

The floating-point value r and the quantized integer q are related by

$$r = S(q - Z).$$

Given the endpoints of the floating-point range and the integer range, we solve

$$\begin{aligned} r_{\min} &= S(q_{\min} - Z), \\ r_{\max} &= S(q_{\max} - Z). \end{aligned}$$

This determines S and Z . In the lecture example, for a 2-bit range $[-2, 1]$, solving the system yields $Z = -1$.

4 Linear Quantization for Matrix Multiplication

The main advantage of linear quantization over k-means quantization is that it can convert inference-time matrix multiplication into mostly integer arithmetic.

Consider the floating-point matrix multiplication

$$Y = WX.$$

Quantize W , X , and Y with their own scales and zero points:

$$\begin{aligned} W &= S_W(Q_W - Z_W), \\ X &= S_X(Q_X - Z_X), \\ Y &= S_Y(Q_Y - Z_Y). \end{aligned}$$

Substituting these into $Y = WX$ and solving for Q_Y gives

$$Q_Y = \frac{S_W S_X}{S_Y} (Q_W Q_X - Z_W Q_X - Z_X Q_W + Z_W Z_X) + Z_Y.$$

The factor

$$\frac{S_W S_X}{S_Y}$$

is known before inference and can be precomputed. Empirically, this factor often lies in $(0, 1)$, so it can be represented as

$$\frac{S_W S_X}{S_Y} = 2^{-n} M_0, \quad M_0 \in [0.5, 1),$$

which makes it possible to implement the scaling using fixed-point multiplication and bit shifts instead of floating-point multiplication.

The remaining terms mostly involve integer tensors and integer zero points. The matrix multiplication can therefore run on integer multiply-accumulate units. Additions and subtractions are typically accumulated in 32-bit integers to avoid overflow.

4.1 Symmetric Weights

Neural network weights are often approximately symmetric around zero. Under symmetric weight quantization, the weight zero point is set to

$$Z_W = 0.$$

This removes all terms involving Z_W and simplifies the output equation to

$$Q_Y = \frac{S_W S_X}{S_Y} (Q_W Q_X - Z_X Q_W) + Z_Y.$$

This simplification reduces runtime operations and makes integer-only inference easier to implement.

4.2 Comparison with K-Means Quantization

Method	Storage Savings	Compute Savings
K-Means Quantization	Yes, stores indices	No, still uses floating-point compute
Linear Quantization	Yes	Yes, enables integer arithmetic

INT8 linear quantization illustrates the accuracy–latency tradeoff. In the lecture examples, ResNet-50 drops from 76.4% floating-point accuracy to 74.9% 8-bit accuracy, while Inception-V3 drops from 78.4% to 75.4%. The benefit is lower latency and better hardware efficiency through integer-only execution.

5 Quantization Granularity

Quantization can be applied at different granularities. Finer granularity usually improves accuracy but increases metadata storage and kernel complexity.

Type	Description	Precision	Efficiency
Per-tensor	One shared scale and zero point for the whole tensor. Vulnerable to outliers.	Lowest	Highest
Per-channel	Separate quantization parameters for each channel.	Medium	Balanced
Group quantization	Channels or weights are divided into smaller groups, each with its own parameters.	Highest	Lowest

At the extreme, assigning one scale and one zero point per element is essentially equivalent to no quantization, because every value can be represented too flexibly.

5.1 Per-Tensor vs. Per-Channel Quantization

In per-tensor quantization, a few outlier channels can force the global scale to become very large. A large scale means a large quantization step size, causing small values in other channels to round to zero and increasing quantization error.

Per-channel quantization fixes this by assigning separate scales to different channels. Since weights are often centered around zero, the zero point is commonly set to zero. In the lecture example, per-tensor quantization produced error 2.28, while per-channel quantization reduced it to 2.08. Although the absolute difference is only 0.20, it can be the difference between a functioning model and model collapse.

5.2 Group Quantization

Group quantization divides weights into small local groups, such as blocks of 64 or 128 weights, and assigns each group its own scale and zero point. This allows the scale to fit each local neighborhood more tightly, reducing quantization error.

The disadvantage is implementation complexity. Kernels must know which group each value belongs to, load the correct scale and zero point from memory, and apply different equations for different groups. This is a major systems challenge: group quantization improves model quality but complicates kernel design. Recent low-precision implementations, such as FP4 inference, often rely on group quantization.

6 Current Quantization Practice

A common training and deployment pattern is:

- During training, models may use FP8 for many operations, while precision-sensitive operations such as softmax are upcast to BF16 or FP32.
- After training, post-training quantization derives scales and zero points manually or using calibration data.
- During inference, weights may be stored in FP4, together with scales and zero points, and recovered to a higher precision format such as FP8 for computation.

Direct training in FP4 is generally unstable, so FP4 is usually used as a post-training inference format rather than the main training format.

7 Mixed Precision Quantization and Training

Uniform quantization uses the same bit-width across all layers or operations. Mixed precision instead assigns different precisions to different components, based on their sensitivity.

The motivation is that not all operations tolerate low precision equally:

- **Sensitive operations:** normalization, softmax, and parameter accumulation often require high precision.
- **Robust operations:** large linear layers and matrix multiplications can usually tolerate lower precision.

Mixed precision can reduce the average bit-width and improve speed while preserving accuracy. However, the design space is large. If each layer has 8 choices for weight precision and 8 choices for activation precision, then each layer has 64 configurations. A model with n layers has 64^n possible configurations, so automated search is often needed.

7.1 Operation Sensitivity

Softmax is sensitive because it computes exponentials, sums, and divisions. In low precision, small values may underflow and large exponentials may overflow. Normalization also involves sums and divisions, making it numerically sensitive.

Parameter updates are also sensitive. For example,

$$\text{Param} \leftarrow \text{Param} + \sum_t \text{grad}_t$$

can lose information if small gradients are accumulated into large parameter values using too few bits. With optimizers such as Adam, the update also depends on first and second moment estimates, which are typically stored in high precision.

7.2 A Standard 16–32 Mixed-Precision Pipeline

A widely used recipe keeps a master copy of weights in FP32 while using FP16 for the heavy forward and backward computations.

1. Store a master copy of weights in FP32.
2. At the start of a training step, cast the FP32 weights down to FP16.
3. Run the forward pass with FP16 weights and FP16 activations.
4. Run the backward pass using FP16 activation gradients and FP16 weights.
5. Compute FP16 weight gradients.
6. Upcast the gradients to FP32 and update the FP32 master weights.

This works because the slow, FLOP-heavy operations are mostly matrix multiplications, which are relatively robust to lower precision. Sensitive but cheaper operations, such as softmax and parameter updates, are performed in higher precision.

The backward pass requires about twice the FLOPs of the forward pass, because it must compute gradients with respect to both activations and weights.

8 Memory Cost of Mixed-Precision Training

Mixed precision saves compute but still requires multiple copies of model states. With Adam, the common rule of thumb is

$$4N + 2N + 2N + 4N + 4N = 16N,$$

where N is the number of parameters. The terms correspond to:

Component	Memory
FP32 master weights	$4N$ bytes
FP16 weight gradients	$2N$ bytes
FP16 running weights	$2N$ bytes
Adam first moment	$4N$ bytes
Adam second moment	$4N$ bytes
Total	$16N$ bytes

Thus, Adam-based mixed-precision training requires roughly 16 bytes per parameter before counting activations, temporary buffers, and communication buffers. For a 10-billion-parameter model, this lower-bound estimate is about 160 GB. For GPT-3 175B, the corresponding components are approximately:

- FP32 master weights: $4 \times 175 \text{ GB} = 700 \text{ GB}$.
- FP16 gradients: $2 \times 175 \text{ GB} = 350 \text{ GB}$.
- FP16 running weights: $2 \times 175 \text{ GB} = 350 \text{ GB}$.
- Adam moments: $8 \times 175 \text{ GB} = 1400 \text{ GB}$.

The optimizer states dominate memory consumption, making memory optimization a central problem in large-scale training.

9 Next-Token Prediction and Language Modeling

Language models are probabilistic next-token predictors. Given a prefix, a model predicts

$$P(x_{t+1} \mid x_1, \dots, x_t).$$

The joint probability of a sequence is factorized using the chain rule:

$$P(x_{1:T}) = \prod_{t=1}^T P(x_{t+1} \mid x_1, \dots, x_t).$$

Training maximizes the likelihood of observed text, or equivalently minimizes cross-entropy loss. Although many architectures can model sequences, such as RNNs, CNNs, and Seq2Seq models, Transformers dominate modern LLMs due to parallelism and long-range dependency handling.

10 Attention and Transformer Blocks

10.1 Scaled Dot-Product Self-Attention

Given queries, keys, and values

$$Q, K, V \in \mathbb{R}^{T \times d},$$

self-attention is defined as

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d}} \right) V.$$

The score between a query q and key k_i is

$$s_i = \frac{qk_i^\top}{\sqrt{d}}.$$

The final output is a weighted sum of value vectors. The $\sqrt{d}$ scaling prevents dot products from becoming too large and saturating the softmax.

10.2 Multi-Head Attention and Grouped Query Attention

Multi-head attention runs several attention heads in parallel. Each head has its own learned projections and can specialize in different patterns, such as syntax, positional structure, or coreference. The head outputs are concatenated and projected back to the model dimension.

In standard multi-head attention, every query head has its own key and value heads, which creates a large KV cache during inference. Grouped Query Attention (GQA) reduces this cost by letting multiple query heads share key and value heads. The spectrum is:

$$\text{MHA} \rightarrow \text{GQA} \rightarrow \text{MQA},$$

where MHA has unique K, V per head, GQA shares K, V within groups, and MQA shares one K, V set across all query heads.

10.3 Transformer Block Structure

For input hidden states X , a simplified Transformer block is

$$\begin{aligned} Z &= \text{SelfAttention}(XW_q, XW_k, XW_v), \\ Z &= \text{LayerNorm}(X + Z), \\ H &= \text{LayerNorm}(\text{FFN}(Z) + Z). \end{aligned}$$

The two main sub-layers are multi-head self-attention and a position-wise feedforward network. Each is wrapped with a residual connection and normalization.

The feedforward network often takes the form

$$\text{FFN}(Z) = f(ZW_1)W_2,$$

where f is a non-linear activation. The original Transformer used ReLU, while modern LLMs often use gated variants such as SwiGLU.

10.4 Masked Causal Attention

Autoregressive models generate tokens from left to right, so a token must not attend to future tokens. Causal masking enforces this by adding $-\infty$ to upper-triangular entries of the attention score matrix before the softmax.

Masking must happen before the softmax. If future positions were zeroed out after the softmax, the remaining weights would no longer sum to 1. Pre-softmax masking ensures that future positions receive probability approximately zero and the valid positions are properly normalized.

11 Functional Roles of Attention and MLPs

Attention and MLP layers play different functional roles:

- **Attention** routes and aggregates information across token positions. It supports language understanding, syntax, coreference, and long-range dependencies.
- **MLP/FFN** layers store factual associations and transform token-wise representations. They can be viewed as a form of learned key-value memory.
- **Residual streams** act as shared communication channels that both sub-layers read from and write to.

Both attention and MLPs are expensive. This motivates research into sparse Mixture-of-Experts FFNs, approximate attention, and linear attention.

12 Serving and Inference

Serving means running a trained model efficiently for many users. Autoregressive inference has two phases:

- **Prefill**: process the full prompt in parallel. This phase is usually compute-bound.
- **Decode**: generate one token at a time. This phase uses a KV cache and is often memory-bandwidth-bound.

Important serving optimizations include:

- **KV cache**: avoids recomputing keys and values for previous tokens.
- **Continuous batching**: dynamically batches active requests to improve GPU utilization.
- **FlashAttention**: reorders attention computation to avoid materializing the full $T \times T$ attention matrix.
- **Quantization**: reduces memory footprint and memory bandwidth pressure.
- **Speculative decoding**: uses a small draft model to propose tokens and a larger model to verify them in parallel.

13 Computing Components in LLMs

Modern LLMs are mainly built from stacked Transformer decoder layers. Major components include self-attention, RMSNorm or LayerNorm, residual connections, MLPs, nonlinear activations, embeddings, final linear projection, softmax, and cross-entropy loss.

Self-attention and MLPs are usually the slow components because they involve large matrix multiplications. Normalization, residual additions, activations, word embeddings, and position embeddings are conceptually simple and lower in FLOPs, but normalization can still be slow in practice because it requires reading and writing large activation tensors.

During training, the full sequence is known, so all positions can be processed in parallel under a causal mask. The loss is computed at every token position and averaged or summed over the sequence.

14 Parameter Counts

Estimating parameter counts helps estimate memory cost and communication cost.

14.1 Embedding Layer

If the vocabulary size is V and the hidden dimension is H , the embedding table has shape

$$V \times H,$$

so it contains VH parameters.

14.2 Normalization Layers

LayerNorm or RMSNorm typically has a learnable scaling vector of size H . Compared with attention and MLP layers, normalization has very few parameters.

14.3 Self-Attention

Self-attention uses query, key, value, and output projection matrices:

$$W_q, W_k, W_v, W_o.$$

If each matrix is approximately $H \times H$, the attention module contains about

$$4H^2$$

parameters per layer.

14.4 Feedforward / SwiGLU MLP

Modern LLMs often use SwiGLU, which has gate, up, and down projections. If the hidden dimension is H and the intermediate size is I , then the parameter count is approximately

$$3HI.$$

Because I is usually much larger than H , the MLP often contributes a large fraction of total parameters.

14.5 Final Linear Projection

The final linear layer maps hidden states back to vocabulary logits. Its shape is

$$V \times H.$$

Some models tie the input embedding matrix and output projection matrix to reduce parameter count.

15 Scaling Bottlenecks

Scaling LLMs introduces three main bottlenecks: compute, memory, and communication.

15.1 Compute Bottleneck

Attention and MLP matrix multiplications dominate compute. Training FLOPs are often estimated by the rule

$$\text{FLOPs} \approx 6 \times \text{parameters} \times \text{tokens}.$$

Larger hidden dimensions, more layers, longer sequences, and larger batch sizes all increase compute cost.

15.2 Memory Bottleneck

Training memory is much larger than inference memory because it must store parameters, gradients, optimizer states, activations, temporary buffers, and communication buffers. Adam optimizer states are especially expensive. During inference, the KV cache can become the dominant memory cost, growing with sequence length, batch size, number of layers, and attention head dimensions.

15.3 Communication Bottleneck

Large LLMs are trained and served across multiple GPUs. Communication appears in several forms:

- **Data parallelism:** GPUs process different batches and synchronize gradients.
- **Tensor parallelism:** large matrix multiplications are split across GPUs, requiring partial results to be exchanged.
- **Pipeline parallelism:** different layers are placed on different GPUs, so activations must be passed between pipeline stages.

If communication is too high, GPUs spend time waiting for data transfer instead of performing computation.

16 Connection Back to Quantization and Mixed Precision

Quantization and mixed precision reduce memory usage, bandwidth pressure, and sometimes compute cost. Lower precision formats include FP16, BF16, FP8, INT8, INT4, and FP4. Matrix multiplication is usually robust to lower precision, while softmax, normalization, and optimizer updates often require higher precision.

Per-channel and group quantization improve accuracy by using more local scales, but they increase metadata storage and kernel complexity. Mixed precision provides another path: use low precision where the model is robust and high precision where numerical stability is required.

17 Takeaways

- K-means quantization saves storage but does not directly accelerate computation.
- Linear quantization enables integer arithmetic, which can speed up inference.
- Scale and zero point define the mapping between floating-point and integer values.
- Symmetric weight quantization often sets $Z_W = 0$, simplifying integer matmul.
- Finer quantization granularity improves accuracy but increases kernel complexity.
- Mixed precision keeps sensitive operations in high precision and runs heavy matrix multiplications in lower precision.
- Adam-based mixed-precision training requires about $16N$ bytes for parameters and optimizer states.
- Transformer LLMs are bottlenecked by compute, memory, and communication; KV cache is a major inference memory bottleneck.